

Paper Review & Pipeline Integration Notes

Compliance-to-Code (Li et al., 2025) and APOLLO (Ospanov et al., NeurIPS 2025)

Internal Reference — SEBI ICDR Formal Verification Pipeline

March 2026

Paper 1: Compliance-to-Code (arXiv 2505.19804v3)

What it does

Compliance-to-Code introduces a large-scale Chinese financial regulation dataset (1,159 annotated clauses from 361 Beijing Stock Exchange regulations) and a four-module pipeline called FinCheck that converts regulation text into executable Python compliance-checking code. The central claim is that going directly from regulation text to code in one shot is too hard; an intermediate structured representation dramatically improves code generation accuracy.

The Compliance Unit (CU) — core data structure

A *Compliance Unit* is the minimal atomic regulatory statement, decomposed into four fields:

- **Subject** — the entity legally bound by the rule (e.g. “listed company”, “controlling shareholder”)
- **Condition** — the trigger scenario or prerequisite (thresholds, temporal specs, exception clauses)
- **Constraint** — the actual obligation, prohibition, or requirement (what must/must not be done)
- **Contextual Information** — definitions, calculation methodologies, cross-references needed for correct interpretation

CUs also carry *Inter-Unit Relations* encoding cross-references between clauses: **refer to** (one CU needs another for interpretation), **exclude** (one CU negates another), **only include** (logic restricted to another CU), and **should include** (compliance depends on satisfying another CU).

FinCheck pipeline

1. **Structure Predictor** — fine-tuned LLM extracts CUs and inter-unit relations from raw regulatory text
2. **Code Generator** — takes structured CUs as input (not raw text) and generates executable Python

3. **Information Retriever** — fetches relevant company data to feed into the generated code
4. **Report Generator** — executes code against company data and summarises the outcome with a traceable explanation

Key experimental results

- Direct Regulation-to-Code (R2C, no intermediate structure): **52.9% Pass@1**
- With gold CU structures (R2S $\rightarrow$ S2C): **61.0% Pass@1** — a +8pp gain from structuring alone
- With gold structures + explicit reasoning steps: **78.1% Pass@1** — the best result
- On *difficult* multi-condition rules: zero-shot R2C gets **0% Pass@1**; gold structures + CoT gives **62%**
- One-shot CoT beats zero-shot dramatically on hard cases; few-shot examples are essential for complex rules
- SFT can hurt capable models (Qwen3-8B drops from 60% to 54% after SFT); prompt engineering beats fine-tuning for strong base models
- DeepSeek-R1-0528 achieves **73.7% Pass@1** without any fine-tuning, best overall
- **45% of all errors** come from upstream CU extraction errors propagating into code generation

Paper 2: APOLLO (Ospanov et al., NeurIPS 2025)

What it does

APOLLO (Automated PrOof repair via LLM and Lean cOllaboration) is a modular, model-agnostic framework for Lean 4 theorem proving. Its core insight: LLMs get the high-level proof structure right but fail at closing fine-grained subgoals. Rather than discarding a failing proof and regenerating from scratch thousands of times, APOLLO treats the failing proof as a scaffold and surgically repairs it using compiler feedback.

The four-module pipeline

1. **Syntax Refiner** — regex-based fixes for common LLM syntax errors: Lean 3 keywords (`from` $\rightarrow$ `:= by`), misplaced brackets, missing delimiters. Primarily useful for general-purpose models not trained on Lean 4; dedicated prover models rarely trigger it.
2. **Sorrifier** — compiles the failing proof through the Lean REPL, identifies exactly which blocks are failing, and replaces them with `sorry` placeholders via one of three strategies: line removal, block removal, or `sorry` insertion. After this step the proof compiles, with each `sorry` marking a known unsolved sub-goal.

3. **Auto Solver** — attempts to close each `sorry` automatically using Lean’s built-in tactics (`nlinarith`, `ring`, `simp`, `norm_num`, `omega`, `hint`, etc.). For arithmetic and algebraic goals this often succeeds without any LLM involvement.
4. **Proof Assembler + recursive LLM re-invocation** — any remaining `sorry` blocks are extracted as standalone sub-lemmas with their local proof context and passed back to the LLM individually. Each sub-lemma is then passed through the full pipeline recursively to depth r . Repaired sub-proofs are spliced back and the full proof is re-verified.

Key experimental results

- **84.9% on miniF2F** (sub-8B models, NeurIPS 2025 state of the art) with sampling budget under 100
- Goedel-Prover-SFT: base accuracy 64.7% at 25,600 samples; with APOLLO: **65.6% at only 362 samples** (two orders of magnitude reduction)
- o3-mini / o4-mini: base 3–7%; with APOLLO: **40–47%** — no fine-tuning required
- Auto Solver alone: +0.4% gain; LLM Re-invoker alone: +6.2%; **both together: +11.9%**
- Proofs repaired by APOLLO have 2–4 $\times$ longer proof lengths than base model proofs — APOLLO unlocks deep multi-step proofs the base model cannot handle in one shot
- Even at recursion depth $r = 5$, only 300–400 LLM calls on average — order of magnitude below naive sampling

The ablation result most relevant to us

The ablation (Table 4 in paper) shows that Syntax Refiner and Auto Solver in isolation provide minimal gain. The combination of Auto Solver + LLM Re-invoker is where the leverage comes from: the Auto Solver closes cheap goals (arithmetic, ring, `norm_num`) at zero LLM cost, concentrating the LLM budget on genuinely hard sub-goals. This is exactly the pattern we should replicate.

Ideas to Borrow: Compliance-to-Code

Idea 1: Adopt the CU schema for our `maps_to` structure (Phase 1 — immediate)

Our current `maps_to` has `field`, `type_hint`, and `constraints_text`. This conflates what the Compliance-to-Code paper cleanly separates. We should extend `maps_to` to carry the four CU components explicitly:

- **subject**: which entity the rule applies to (“issuer”, “promoter”, “lead manager”)
- **condition**: the trigger or prerequisite (“in each of the preceding three years”, “if net profit < 0 ”)
- **constraint**: the actual obligation (“net tangible assets ≥ 3 crore”, “shall not disclose”)

- **contextual_info**: calculation basis, cross-references (“restated and consolidated”, “as per Schedule III”)

This structured representation then feeds into Lean code generation instead of raw clause text, directly paralleling their $R2S \rightarrow S2C$ two-stage result (52.9% $\rightarrow$ 78.1% Pass@1).

Idea 2: Two-stage generation for `llm_generate_lean.py` (Phase 3a — immediate)

Instead of: *raw JSONL rules* $\rightarrow$ *Lean check function*

Do: *raw JSONL rules* $\rightarrow$ *CU structured representation* $\rightarrow$ *Lean check function*

The intermediate CU step forces the LLM to correctly identify what the *condition* is versus what the *constraint* is before attempting to write a Lean predicate. The Compliance-to-Code paper’s most important finding is that this separation is what causes the jump from 53% to 78% on hard multi-condition rules. Our hardest Lean generation failures are exactly this conflation of condition and constraint.

Idea 3: Inter-Unit Relations for ICDR cross-references (Phase 1 — near-term)

ICDR regulations frequently reference each other (“subject to Regulation 6(1)...”, “as defined in Schedule II...”). Our pipeline currently treats every rule as independent. The four CU relation types give us a vocabulary to annotate these dependencies:

- **refer to**: our check function must call or import a definition from another rule
- **should include**: our Lean check function for rule A must also invoke rule B’s check
- **exclude**: if condition of rule A holds, rule B does not apply

This directly enables composite eligibility checks that currently cannot be expressed.

Idea 4: Adopt their code generation prompt template (Phase 3a — immediate)

Their instruction-tuning prompt (Figure 4) enforces a fixed five-step skeleton for every generated function: (1) initialise marker columns, (2) subject validation, (3) condition validation, (4) constraint validation, (5) return result. We should adopt an analogous skeleton for our Lean check functions:

1. Declare the theorem signature (issuer type, output Bool)
2. Check subject applicability (does this rule apply to this issuer category?)
3. Evaluate condition (is the trigger met?)
4. Evaluate constraint (does the issuer satisfy the obligation?)
5. Return compliance verdict with a traceable `failReason`

Giving the LLM a scaffold rather than asking it to invent structure reduces hallucinated field names and malformed Lean idioms.

Idea 5: Pass@k evaluation framework (Phase 6 — near-term)

They use Pass@1 and Pass@5 with unit tests as their primary metric. We have no such framework; we only check whether generated Lean compiles. We should define a test suite of (issuer instance, expected compliance verdict) pairs per rule, and measure Pass@1 (correct verdict on first attempt) and Pass@5. This immediately tells us which rules our Lean generation is failing on and why.

Idea 6: Reasoning-augmented generation (Phase 3a — immediate)

Their best result (78.1% Pass@1) uses *Structure-Reasoning-to-Code*: the model gets the CU structure *plus* an explicit step-by-step reasoning trace before generating code. For our Lean generation prompt, this means including a natural language “proof sketch” of the compliance logic before asking for the Lean check function. This is independently confirmed by Seed-Prover 1.5’s informal-then-formal architecture and by ARCV’s chain-of-thought findings.

Ideas to Borrow: APOLLO

Idea 7: APOLLO-style repair loop in `verify_one.py` (Phase 6 — highest priority)

This is the single most impactful addition. Replace the current “generate once, run lake build” with a bounded iterative repair loop:

1. Call `llm_generate_lean.py` to produce an initial Lean proof attempt
2. Run the Lean REPL on the generated file; collect structured error messages with line numbers
3. **Syntax Refiner**: fix common Lean 4 idiom errors (our `postprocess_rules_and_fields.py` already does part of this at the JSON level; extend it to the Lean level)
4. **Sorrifier**: parse the proof, locate each failing block, replace with `sorry`
5. **Auto Solver**: for each `sorry` goal, try `omega`, `decide`, `simp`, `norm_num`, `ring` in sequence; close what you can for free
6. **LLM Re-invoker**: for each remaining `sorry`, extract its local context (hypotheses, definitions, proven lemmas so far) and give it to the LLM as a standalone sub-problem
7. **Proof Assembler**: splice repaired sub-proofs back; run `lake build` to verify
8. Repeat up to $r = 3\text{--}5$ rounds; quarantine persistent failures

Expected impact (from APOLLO’s o3-mini/o4-mini result): general-purpose models jump from 3–7% to 40–47% proof success *without any fine-tuning*. This is the architecture upgrade that makes our Phase 6 actually work in production.

Idea 8: MA-LoT corrector split in `llm_generate_lean.py` (Phase 3a — immediate)

Currently `llm_generate_lean.py` uses one prompt for both generation and correction. Separate into two calls:

- **Generator prompt:** receives rule text + CU structure + issuer schema; produces initial Lean proof
- **Corrector prompt:** receives *only* the Lean REPL error message and the failing sub-goal; produces only the fix for that sub-goal

The corrector never sees the full rule text — it only sees the failing proof state and the error. This is both cheaper and more targeted than full regeneration.

Idea 9: Use the Lean REPL (not lake build) as the feedback oracle (Phase 4/6 — immediate)

APOLLO uses the Lean REPL (`leanprover-community/repl`) rather than a full `lake build` for per-attempt feedback. The REPL is much faster for checking individual proof attempts interactively and returns structured error messages with exact line numbers. We should wire `schema_reconcile.py` and `verify_one.py` to the REPL for the repair loop, reserving `lake build` for final verification only.

Idea 10: Sub-lemma isolation maps onto our compliance rule hierarchy (Phase 6 — near-term)

APOLLO’s *sorrify* → sub-lemma recursion maps naturally onto how ICDR eligibility rules are structured. A composite rule like “issuer is eligible if: (a) net tangible assets ≥ 3 cr **and** (b) operating profit ≥ 15 cr **and** (c) not debarred” can be *sorrified* into three independent sub-lemmas, each of which is independently dischargeable by `omega/decide` (for the numeric comparisons) or the LLM (for the boolean flags). This is directly analogous to APOLLO’s proof decomposition and should be implemented as the standard pattern for composite eligibility check functions.

Idea 11: Auto Solver tactic precedence for compliance lemmas (Phase 6 — immediate)

Based on APOLLO’s Auto Solver design, we should implement a fixed tactic waterfall for our compliance sub-goals. Recommended order for ICDR-type lemmas:

1. `decide` — for fully decidable propositions (boolean flags, exact equality checks)
2. `omega` — for linear arithmetic over `Nat` and `Int` (threshold comparisons)
3. `norm_num` — for numeric literal computations
4. `simp` + `List.all`, `List.any` lemmas — for list membership and multi-year checks
5. LLM invocation — only for goals that survive the above

Most ICDR compliance checks are decidable propositions or linear arithmetic — the vast majority should be closeable by steps 1–3 without any LLM involvement.

Idea 12: Recursion depth as a tuneable budget parameter (Phase 6 — near-term)

APOLLO’s recursion depth r directly trades off sampling cost against accuracy. From their Figure 5: the largest accuracy gain for general-purpose models occurs between $r = 0$ and $r = 1$ (syntax fixes + Auto Solver). Beyond $r = 3$, gains plateau for most theorems. For our pipeline, start with $r = 3$ as default. Log which rules require which depth to close; rules that persistently require $r > 3$ are candidates for manual review or quarantine.

Combined Integration Priority

Idea	Phase	Expected impact
7. APOLLO repair loop in <code>verify_one.py</code>	6	Critical / highest
2. Two-stage generation (CU $\rightarrow$ Lean)	3a	High
8. MA-LoT corrector split	3a	High
1. CU schema in <code>maps_to (llm_extract_rules)</code>	1	High
11. Auto Solver tactic waterfall	6	High
9. Lean REPL as feedback oracle	4/6	High
6. Reasoning-augmented generation prompt	3a	Medium
4. Fixed Lean check function skeleton	3a	Medium
3. Inter-Unit Relations for cross-refs	1	Medium
5. Pass@k evaluation framework	6	Medium
10. Sub-lemma isolation for composite rules	6	Medium
12. Recursion depth as budget parameter	6	Low / config

The single most important observation across both papers: both independently confirm that going from natural language $\rightarrow$ code (or $\rightarrow$ formal proof) in one shot is the wrong architecture. The right pattern is always: *decompose first, then generate, then repair with compiler feedback*. Compliance-to-Code proves this for the regulation $\rightarrow$ code direction. APOLLO proves it for the code $\rightarrow$ formal proof direction. Our pipeline needs both.

What We Do That Neither Paper Does

Neither paper combines both directions. Compliance-to-Code uses Python (probabilistic verifier; no formal soundness). APOLLO uses mathematics (Mathlib; no regulatory domain). We are the first to apply the full chain: *regulation text $\rightarrow$ structured CU decomposition $\rightarrow$ Lean formal representation $\rightarrow$ compiler-guided repair loop $\rightarrow$ formally verified compliance verdict*. The Lean kernel is a formally sound oracle; neither paper has this. Our pipeline, if completed, is the first formally verified regulatory compliance system for financial documents. This gap is the basis for a publication contribution at ICAIL, FinNLP, or a formal methods venue.